

# ParaSol

## Solist-AI™ MCU を SPI ペリフェラル・デバイス化するファームウェア

Ver.1.0 - Jun 2026

Product manual

## 1 概要

ParaSol はローム株式会社のマイコン ML63Q2500 シリーズに搭載されている Solist-AI™ の機能を、ペリフェラル・デバイスとして利用可能にするファームウェアです。SPI 通信に対応し、ホスト CPU のメーカーや型式に依存せず、データを与えるだけで Solist-AI™ の機能を利用できます。オプション機能の CRC による誤り検出を使用することで、通信データの誤り・破損を検出し、信頼性の高いデータ転送を実現できます。待機時の低速クロック動作や省電力スタンバイ機能を利用し、高い省電力性を実現できます。

ParaSol は株式会社データ・テクノのブレイクアウトボード "AIBBY" 向けに開発されています。[10章](#)では、同社の評価ボード DT-EBML63Q2557やローム株式会社のリファレンスボード RB-D63Q2557TB64をはじめ、48ピンデバイスへ移植するために必要な I/O ポートの割り当て変更方法も紹介しています。

## 2 特徴

- Solist-AI™ API の全機能に対応
- SPI スレーブ動作 モード 0 に対応、最大クロック速度 12MHz
- 外部回路を必要としないデュアルポート通信機能
- READY・電源状態通知信号
- CRC32 による誤り検出機能
- 送受信合計 16K バイトの大容量通信バッファ
- 待機時は低速クロックで動作、省電力スタンバイ機能を搭載
- 株式会社データ・テクノの "AIBBY" に対応

## 3 応用例

- Arduino や M5Stack などと組み合わせた試作 AI システム
- 高性能 CPU と ParaSol のデュアル CPU システム
- 低速 CPU 向けの FFT 処理システム
- USB-SPI 変換 I/F 等を経由した PC 接続用 AI デバイス
- デュアルポート機能を活用した Solist-AI™ SIM データのインポート・動作検証システム

## 4 動作概要

図 1・図 2 に ParaSol の動作ブロック図を示します。

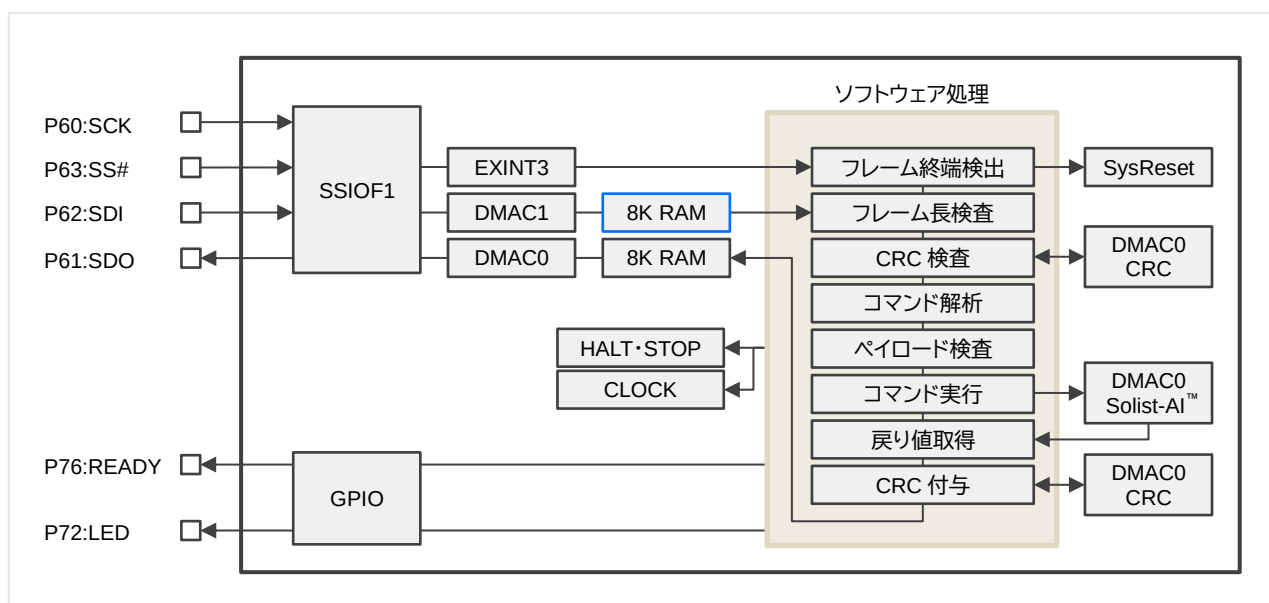


図 1: 動作ブロック図 シングルポート構成

## 4 動作概要 (続き)

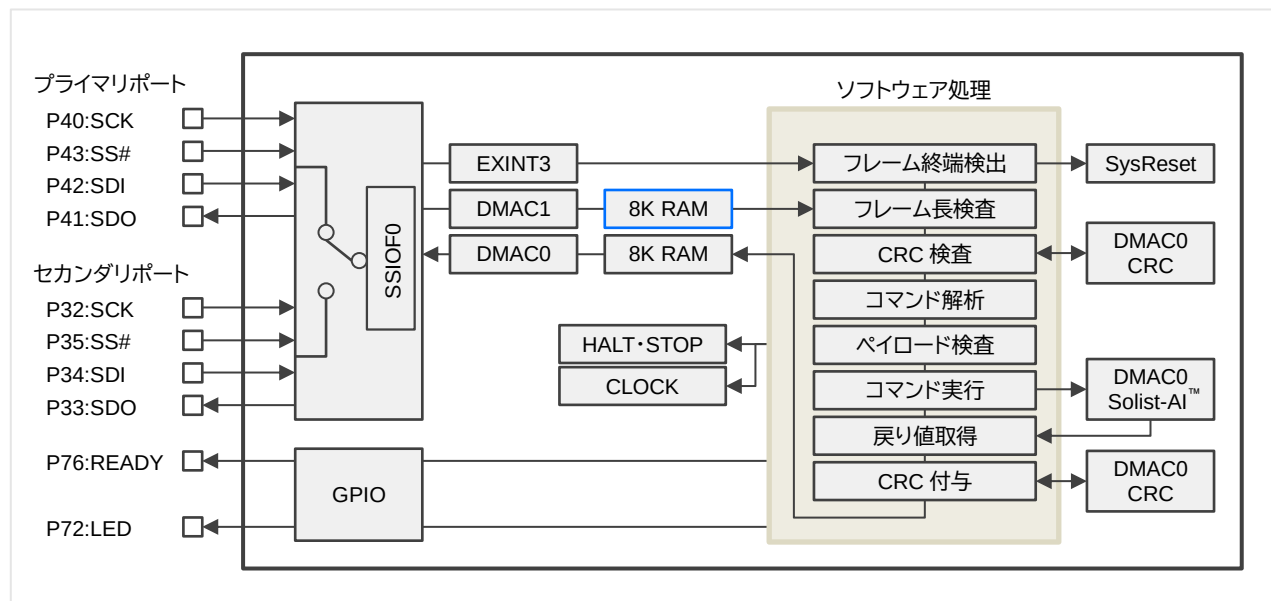


図 2: 動作ブロック図 デュアルポート構成

### 【シングルポート構成】

- SPI スレーブデバイス、モード 0、SS#は負論理、最大クロック速度は12MHz、送受信バッファは各 8K バイト。
- SS#の立ち上がりエッジで SPI トランシーバを停止し、高速クロックへ移行、受信データの処理を開始する。
- データの処理が完了後、送信バッファへデータをセット、SPI トランシーバを再開、低速クロックへ移行。
- READY 状態通知信号は、High: 通信可能・省電力スタンバイ状態 Low: コマンド処理中
- 動作確認 LED は、電源 ON 中に点灯、省電力スタンバイ状態に消灯。
- データを送受信せずに SS#を 4 回クロッキングすると、システムリセットを実行し全てのデータを初期化、LED が短時間消灯する。
- 省電力スタンバイ状態から復帰するには、SS#を 1 回クロッキングする。CPU はスタンバイ直前の状態から動作を再開する。

### 【デュアルポート構成】

デュアルポート構成でプログラムがコンパイルされている場合は、シングルポート構成の動作に加えて下記の動作が加わる。

- システムリセット後はプライマリポートで通信を開始する。
- Change Access Port コマンドでプライマリ・セカンダリポートを選択する。選択されていないポートの SDO は Hi-Z になる。
- セカンダリポートが選択されている場合、SS#によるシステムリセット・スタンバイ復帰は両方のポートで実行可能。

## 5 通信データのフレーム構造

【受信】 <Command> <LEN> [Payload] [CRC] [Padding]

【送信】 <Status> <LEN> [Payload] [CRC] [Padding]

<>は必須のヘッダフィールド、[Payload]は可変長、[CRC]は設定コマンドにより使用を選択可能。最大フレーム長は 8191 バイト。任意の場所で SS#を High にすることでデータ転送を中断できる。中断後は送信バッファのデータが破棄される。

データ転送は受信→送信といった交互転送、または、受信／送信の同時通信もサポートする。交互通信で受信する場合、ダミーデータとして 00h または FFh を送信する。同時通信の場合、受信したコマンドに対する結果は、次の通信タイミングに送信する。

送受信のデータ長を一致させるために、適宜パディングが付与される。パディングか否かは LEN フィールドを使って算出できる。

## 6 フレーム構造の詳細

### 【ParaSol が受信するフレーム】

|         |     |         |           |      |     |         |
|---------|-----|---------|-----------|------|-----|---------|
| Command | LEN | Payload | Parameter | Data | CRC | Padding |
|---------|-----|---------|-----------|------|-----|---------|

**Command: 1 バイト**

実行する[コマンド番号](#)。

**LEN: 2 バイト、リトルエンディアン**

Payload の有効長。

CRC 不使用時: 0～8188

CRC 使用時: 0～8184

**Payload: 可変長**

LEN で指定された長さを有効なペイロードとしてコマンドプロセッサに渡す。LEN が 0 の場合、有効なペイロードは存在しない。ペイロードの中には、コマンドごとに役割が変わる固定長のパラメータと、可変長のデータがカプセル化される。

実際のフレーム長がヘッダ 3 バイト + LEN (+ CRC が有効なら 4 バイト) よりも短い場合は、[フレームチェックエラー【4】](#)とする。

**CRC: 4 バイト、ビッグエンディアン**

CRC を使用しない場合は、CRC を省略する。

CRC を使用する場合は、CRC32\_MPEG2 形式の CRC を付与する。対象範囲は Command から Payload 末尾までの全区間。

**Padding:**

CRC 以降の余分なデータは無視する。

### 【ParaSol から送信するフレーム】

|        |     |         |     |         |
|--------|-----|---------|-----|---------|
| Status | LEN | Payload | CRC | Padding |
|--------|-----|---------|-----|---------|

**Status: 1 バイト**

MSB 7: [0] CRC 不使用 [1] CRC 使用

6-0: [結果コード](#)

**LEN: 2 バイト、リトルエンディアン**

Payload の有効長。

CRC 不使用時: 0～8188

CRC 使用時: 0～8184

**Payload: 可変長**

LEN で指定された長さがペイロードとして有効。LEN が 0 の場合、有効なペイロードは存在しない。

**CRC: 4 バイト、ビッグエンディアン**

CRC を使用しない場合は、CRC を省略する。

CRC を使用する場合は、CRC32\_MPEG2 形式の CRC が付与される。対象範囲は Status から Payload 末尾までの全区間。

**Padding:**

CRC 以降の余分なバイトは FFh でパディングされる。

## 7 結果コード一覧

### 【正常終了】

- 0: 戻り値なし。
- 1: 戻り値あり。

### 【フレームチェックエラー】

- 2: 送受信カウントが 8192 バイト目に達した。フレームが長すぎて末尾が切れている可能性がある。
- 3: LEN フィールドの値が不正。8188 または 8184 を超えている。
- 4: 受信したペイロードのバイト数が LEN フィールドの値より少ない。
- 5: CRC が不一致。

### 【コマンド解析エラー】

- 6: コマンド未定義。戻り値: 受信したコマンド番号。(1 バイト)
- 7: パラメータのバイト数が不足している。
- 8: パラメータチェックエラー。戻り値: 問題のあるパラメータ番号。(1 バイト)
- 9: コマンドに渡すデータバイト数が足りない。
- 10: コマンドに渡すデータバイト数が多すぎる。

### 【コマンド実行エラー】

- 11: 事前の初期化・学習・推論コマンドが未実行。
- 12: コマンドが実行エラーを返した。

### 【その他】

- 119: リセット成功。(77h)
- 127: オフライン。

#### ※ 補足 - パラメータ・コマンドについて

フレーム図のペイロードのうち、

パラメータ : コマンドの動作に必要な固定長の値。

一部のパラメータは値の妥当性をチェックする。

データバイト : コマンドで処理する可変長の値。

黄色に塗られている部分がデータバイト。

## 8 動作状態の判定方法

SPI の結果コードや LED・READY 信号を利用して、ParaSol の動作状態を判定することができます。

表 1: 結果コードからわかる ParaSol の動作状態

| 結果コード     | 状態                                                           |
|-----------|--------------------------------------------------------------|
| 0 ~ 126   | ParaSol はコマンド処理を行った。                                         |
| 127 (77h) | コマンド処理中、デュアルポート構成の非アクティブポート、<br>省電力スタンバイ中、再起動中、電源 OFF のいずれか。 |

結果コード【127】は、ParaSol が何らかの原因で応答できない場合に現れるステータスです。いずれの場合も SPI トランスミッタが停止している状態ですので、外部回路で SDO ピンを弱プルアップしてください。

表 2: LED・READY 信号の状態

| LED  | READY | 状態          |
|------|-------|-------------|
| High | High  | コマンド待機中     |
| High | Low   | コマンド処理中     |
| Low  | High  | 省電力スタンバイ状態  |
| Low  | Low   | 再起動中、電源 OFF |

結果コード【127】の時は LED・READY 信号を GPIO で監視すると、ParaSol が応答できない原因を判定できます。

## 9 コマンドリファレンス

### 【コマンド大分類】

|     |                                             |
|-----|---------------------------------------------|
| 00h | NOP (予約)                                    |
| 0xh | <a href="#">システムコマンド</a>                    |
| 1xh | <a href="#">ML ACC コマンド</a>                 |
| 2xh | <a href="#">FFT コマンド</a>                    |
| 3xh | <a href="#">ODL 数値変換コマンド</a>                |
| 4xh | <a href="#">ODL オンデバイス学習コマンド</a>            |
| 5xh | <a href="#">OSUAD オンライン逐次学習型 教師なし学習コマンド</a> |
| 6xh | <a href="#">ODL 重みデータセーブ・リストアコマンド</a>       |
| FFh | NOP (予約)                                    |

### 【コマンド実行の注意点】

- LEN フィールドや複数バイトのパラメータは全てリトルエンディアンであり、CRC のみがビッグエンディアンである。
- フレーム図のペイロード長の数値を「書かれた通り」そのまま送信すると、ビッグエンディアンになるので注意すること。
- フレーム図のペイロード長が LEN と記載されているものは可変長フレームである。
- CRC チェック機能がオフの状態ではペイロード長が 0 のコマンドは、LEN フィールドを省略した 1 バイトのコマンドでも実行できる。
- 型名が記載されていないフィールドは、符号なしの整数が入る。
- 未定義のコマンドは[コマンド解析エラー](#)【6】を返す。

- 戻り値なしと書かれているコマンドは、正常終了、戻り値なし【0】を示す以下のフレームが戻る。

|     |       |     |
|-----|-------|-----|
| 00h | 0000h | CRC |
|-----|-------|-----|

- 大半のコマンドのパラメータは ParaSol で値の妥当性をチェックしてから Solist-AI™ の API へ渡しているが、[ODL Parameters 構造体](#)をはじめ、データそのものはチェックしていない。Solist-AI™ エンジンでは異常な値が入ると、CPU がリセットされることがある。CPU がリセットされると結果コードが【119】になるので、問題があったことを把握できるようになっている。

### 【システムコマンド】

#### -----: System Reset

データを送受信せずに SS# を 4 回連続でクロッキングすると、立ち上がりエッジでシステムリセットを実行する。

リセットは約 0.6 秒かかり、リセット中は LED が消灯する。

パルス幅は 10 $\mu$ s 程度の短時間でよい。

#### 01h: CRC Check Off

|     |       |
|-----|-------|
| 01h | 0000h |
|-----|-------|

CRC有効時

|     |       |            |
|-----|-------|------------|
| 01h | 0000h | B6BC_D187h |
|-----|-------|------------|

戻り値 なし

CRC チェックを無効にする。CRC 有効から無効へ切り替えるときは、CRC として B6BC\_D187h を付ける。

#### 02h: CRC Check On

|     |       |     |
|-----|-------|-----|
| 02h | 0000h | CRC |
|-----|-------|-----|

戻り値 なし

CRC チェックを有効にする。

## 9 コマンドリファレンス (続き)

### 03h: Set Idle Clock

|     |       |          |     |     |    |
|-----|-------|----------|-----|-----|----|
| 03h | 0001h | Clock 1B | CRC | 戻り値 | なし |
|-----|-------|----------|-----|-----|----|

コマンド実行中以外のクロック速度を指定する。コマンドの実行中は最高速の 48MHz に切り替わる。

通信速度を要求されない省電力アプリケーションで消費電流を抑えられる。

Clock の設定値と SPI の通信速度の関係を以下の表に示す。

表 3: 待機クロック速度 設定値

| 設定値       | 待機 CPU 速度 | SPI 最高通信速度 | 消費電流の目安 |
|-----------|-----------|------------|---------|
| 00h (初期値) | 48 MHz    | 12 MHz     | 4.7 mA  |
| 01h       | 24 MHz    | 8 MHz      | 3.1 mA  |
| 02h       | 12 MHz    | 4 MHz      | 1.7 mA  |
| 03h       | 6 MHz     | 2 MHz      | 1.4 mA  |
| 04h       | 3 MHz     | 1 MHz      | 1.2 mA  |
| 05h       | 1.5 MHz   | 500 kHz    | 1.0 mA  |

※ 消費電流の目安は AIBBY を使用し、LED を不点灯の状態で運用した場合の数値である。

※ 最高通信速度のギリギリで通信すると、クロック誤差でデータを取りこぼす可能性がある。

水晶振動子の場合は 200ppm 程度、RC 発振の場合は 2%程度の余裕を持って使うこと。

### 04h: Enter Standby Mode

|     |       |     |     |    |
|-----|-------|-----|-----|----|
| 04h | 0000h | CRC | 戻り値 | なし |
|-----|-------|-----|-----|----|

動作を停止し省電力スタンバイ状態に入る。LED は消灯し、READY 信号は High になる。

SS#を 1 回クロッキングすると、立ち上がりエッジでスタンバイ直前の状態から動作を再開する。

パルス幅は 10 $\mu$ s 程度の短時間でよい。再開処理は約 0.4 秒かかり、LED が点灯し、READY 信号は Low になる。

### 05h: Get Version

|     |       |     |     |     |       |          |          |            |     |
|-----|-------|-----|-----|-----|-------|----------|----------|------------|-----|
| 05h | 0000h | CRC | 戻り値 | 01h | 0004h | Minor 1B | Major 1B | AI Ver. 2B | CRC |
|-----|-------|-----|-----|-----|-------|----------|----------|------------|-----|

ParaSol と Solist-AI™ API のバージョン情報を取得する。

### 06h: Calc Payload CRC

|     |     |         |     |     |     |       |           |     |
|-----|-----|---------|-----|-----|-----|-------|-----------|-----|
| 06h | LEN | Payload | CRC | 戻り値 | 01h | 0004h | Result 4B | CRC |
|-----|-----|---------|-----|-----|-----|-------|-----------|-----|

Payload の CRC を計算する。ビッグエンディアン 4 バイトの CRC が戻る。

### 07h: Readback Payload

|     |     |         |     |     |     |     |         |     |
|-----|-----|---------|-----|-----|-----|-----|---------|-----|
| 07h | LEN | Payload | CRC | 戻り値 | 01h | LEN | Payload | CRC |
|-----|-----|---------|-----|-----|-----|-----|---------|-----|

通信状態の確認に使うコマンド。LEN と Payload をそのまま返す。CRC は再計算される。

LEN に対して Payload が長い場合は、LEN の長さで切り詰められた Payload が戻る。

### 08h: Change Access Port

|     |       |         |     |     |    |
|-----|-------|---------|-----|-----|----|
| 08h | 0001h | port 1B | CRC | 戻り値 | なし |
|-----|-------|---------|-----|-----|----|

アクセスポートを 0 (プライマリ)、1 (セカンダリ) のどちらかに切り替える。

セカンダリポートが選択されている時は、SS#によるシステムリセット・スタンバイ復帰は両方のポートで実行可能。

シングルポート構成でプログラムがコンパイルされている場合、このコマンドは使用できない。

## 9 コマンドリファレンス (続き)

---

### 【ML\_ACC コマンド】

#### 10h: ML\_ACC\_ClearRAM

|     |       |     |
|-----|-------|-----|
| 10h | 0000h | CRC |
|-----|-------|-----|

戻り値 なし

AI 専用メモリをゼロクリアする。

---

### 【FFT コマンド】

#### 20h: FFT\_Initialize

|     |       |         |           |     |
|-----|-------|---------|-----------|-----|
| 20h | 0003h | size 2B | window 1B | CRC |
|-----|-------|---------|-----------|-----|

戻り値 なし

FFT の入力データ数と窓関数テーブルを初期化する。

同じデータ数と窓関数を使用し続けるなら、FFT\_Initialize は一度だけ実行すればよい。

size : FFT 入力データ数 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048 のいずれか。

window : 窓関数 0 (窓関数なし) 1 (ハニング窓)

#### 21h: FFT\_Execute

|     |     |                     |     |
|-----|-----|---------------------|-----|
| 21h | LEN | BF16 data 2B×(size) | CRC |
|-----|-----|---------------------|-----|

戻り値 

|     |     |                       |     |
|-----|-----|-----------------------|-----|
| 01h | LEN | BF16 result 2B+(size) | CRC |
|-----|-----|-----------------------|-----|

FFT の計算を実行する。事前に FFT\_Initialize で入力データ数を宣言する必要がある。

戻り値の数は、DC 成分 1+入力データの半数の周波数成分が戻る。

## 9 コマンドリファレンス (続き)

### 【数値変換コマンド】

#### 30h: ODL\_GenerateRandomNumber

|     |       |         |         |     |
|-----|-------|---------|---------|-----|
| 30h | 0004h | size 2B | seed 2B | CRC |
|-----|-------|---------|---------|-----|

|     |     |     |                |     |
|-----|-----|-----|----------------|-----|
| 戻り値 | 01h | LEN | BF16 2B×(size) | CRC |
|-----|-----|-----|----------------|-----|

指定した個数の乱数配列を発生させる。発生する乱数の範囲は 0～1 の実数。

size は CRC 未使用時 4093、CRC 使用時 4091 要素まで指定可能。

seed は 1～32767。seed が同じなら、生成される乱数の数列は毎回同じになる。

隠し関数の ODL\_SetWeightAlpha や ODL\_SetWeightGamma で使うと思われるが、実際の用途は不明。

#### 31h: ODL\_FixedToBfloat16

|     |     |             |                 |     |
|-----|-----|-------------|-----------------|-----|
| 31h | LEN | Q-Format 1B | int16 2B×(data) | CRC |
|-----|-----|-------------|-----------------|-----|

|     |     |     |                |     |
|-----|-----|-----|----------------|-----|
| 戻り値 | 01h | LEN | BF16 2B×(data) | CRC |
|-----|-----|-----|----------------|-----|

int16 符号付き固定小数点数の配列を bfloat16 の配列へ変換する。

Q-Format には 0～15 で小数点のビット位置を指定できる。Q0 ならば変換後に -32768 ～ 32767 となる。

CRC 不使用時 4093、CRC 使用時 4091 要素まで一度に変換可能。

#### 32h: ODL\_Bfloat16ToFixed

|     |     |             |                |     |
|-----|-----|-------------|----------------|-----|
| 32h | LEN | Q-Format 1B | BF16 2B×(data) | CRC |
|-----|-----|-------------|----------------|-----|

|     |     |     |            |                 |     |
|-----|-----|-----|------------|-----------------|-----|
| 戻り値 | 01h | LEN | Padding 1B | int16 2B×(data) | CRC |
|-----|-----|-----|------------|-----------------|-----|

bfloat16 の配列を int16 符号付き固定小数点数の配列へ変換する。

Q-Format には 0～15 で小数点のビット位置を指定できる。Q0 ならば変換後に -32768 ～ 32767 となる。

戻り値のパディングバイトは 77h が入る。

CRC 不使用時 4093、CRC 使用時 4091 要素まで一度に変換可能。

#### 33h: ODL\_Bfloat16ToFloat

|     |     |            |                |     |
|-----|-----|------------|----------------|-----|
| 33h | LEN | Padding 1B | BF16 2B×(data) | CRC |
|-----|-----|------------|----------------|-----|

|     |     |     |            |                 |     |
|-----|-----|-----|------------|-----------------|-----|
| 戻り値 | 01h | LEN | Padding 1B | float 4B×(data) | CRC |
|-----|-----|-----|------------|-----------------|-----|

bfloat16 の配列を float の配列へ変換する。

引数のパディングバイトの値はノーチェックだが、LEN の長さに含まれる。

戻りのパディングバイトは 77h が入る。

CRC 不使用時 2046、CRC 使用時 2045 要素まで一度に変換可能。



## 9 コマンドリファレンス (続き)

### 【ODL オンデバイス学習コマンド】

オンデバイス学習機能は大きく分けて、教師データ  $t$  を使った【教師あり学習】と、入力データ  $x$  を教師データ  $t$  とする【教師なし学習】の 2 通りがある。各コマンドの利用方法や使用する順番は、ロームのアプリケーションノート「Solist-AI™ 加速度センサを使った異常検知と AI ライブラリ使用方法」や「ML63Q2500 リファレンスソフトウェア スタートアップ マニュアル」を参照していただきたい。

【教師なし学習】を行う場合は、次節の [OSUAD コマンド](#) を使うと教師データ  $t$  の転送を省略できる。

#### 40h: ODL\_SetModelCount

|     |       |           |     |        |
|-----|-------|-----------|-----|--------|
| 40h | 0001h | models 1B | CRC | 戻り値 なし |
|-----|-------|-----------|-----|--------|

使用する AI モデルのインスタンス数を 1～3 で宣言する。

AI エンジンのメモリ不足などで初期化が失敗した場合は、[コマンド実行エラー【12】](#)が戻る。

#### 41h: ODL\_Initialize

|     |       |             |                    |     |        |
|-----|-------|-------------|--------------------|-----|--------|
| 41h | 0015h | instance 1B | ODL_Parameters 20B | CRC | 戻り値 なし |
|-----|-------|-------------|--------------------|-----|--------|

指定したインスタンス番号の AI モデルを初期化する。インスタンス番号は 0～2 で指定する。

初期化はメモリ割り当ての変更を伴うため、若いインスタンス番号から順に実行しないとイケない。

例えばインスタンス 1 を初期化後にインスタンス 0 を初期化すると、インスタンス 1 はデータが破壊された状態になっている。

AI エンジンのメモリ不足などで初期化が失敗した場合は、[コマンド実行エラー【12】](#)が戻る。

ODL\_Parameters 構造体の内容を以下の表に示す。

表 4: ODL\_Parameters 構造体

| 項目                 | 型      | バイト数 | 機能                                                                                                                                                                                                                    |
|--------------------|--------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| inputSize          | uint16 | 2    | 入力ノード数。(1 以上)                                                                                                                                                                                                         |
| hiddenSize         | uint16 | 2    | 隠れ層のノード数。(1 以上)                                                                                                                                                                                                       |
| outputSize         | uint16 | 2    | 出力ノード数。(1 以上)、OSUAD で使用する場合は inputSize と同一にする。                                                                                                                                                                        |
| forgettingFactor   | BF16   | 2    | 逐次学習時の忘却率。範囲は 0.5～2 で、0 に近いほど新しい入力データを優先する。                                                                                                                                                                           |
| activationFunction | uint8  | 1    | 隠れ層を計算する際に使用する活性化関数。<br>(0) Linear function. $f(x) = x$ .<br>(1) Hard Sigmoid function. $f(x) = \max(0, \min(1, 0.2x+0.5))$<br>(2) ReLU function. $f(x) = \max(0, x)$<br>(3) Hard tanh. $f(x) = \max(-1, \min(1, x))$ |
| lossFunction       | uint8  | 1    | Loss の算出に使用する損失関数。学習そのものには影響しない。<br>(0) Mean Absolute Error<br>(1) Mean Squared Error                                                                                                                                 |
| seed               | uint16 | 2    | 重み $\alpha \cdot \gamma$ を生成する乱数のシード値。範囲は 1～32767。<br>Solist-AI™ は Extreme Learning Machine なので、入力ノードと隠れ層の間の重みは乱数が用いられる。                                                                                              |
| scaleAlpha         | BF16   | 2    | 入力ノードと隠れ層の間にある重み $\alpha$ が取りうる乱数の範囲。<br>[-scaleAlpha, scaleAlpha] の一様乱数となる                                                                                                                                           |
| scaleGamma         | BF16   | 2    | Echo State Network の隠れ層ノードのフィードバック重み $\gamma$ が取りうる乱数の範囲。<br>scaleGamma=0 なら、フィードバックのない通常の 3 層ニューラルネットワークである。<br>[-scaleGamma, scaleGamma] の一様乱数となる。                                                                  |
| leakRate           | BF16   | 2    | Echo State Network のリークレート。範囲は 0～1。                                                                                                                                                                                   |
| l2Param            | BF16   | 2    | 過学習防止に使われる L2 正則化項。 $10^{-3} \sim 10^3$ 程度の範囲で使用する。<br>0 なら正則化は無効。                                                                                                                                                    |

## 9 コマンドリファレンス (続き)

hiddenSize・outputSize は AI エンジンのメモリ容量により上限が決められ、以下の式により 128KB 以下でなければならない。  
複数のインスタンスを使用する場合、全ての AI モデルの合計で 128KB の制限が課される。

$$(\text{outputSize} + \text{hiddenSize} \times 2) \times \text{hiddenSize} \times 2 + 16 \times 1024 \leq 131072 \text{ バイト}$$

|   | A      | B      | C                        | D                           |
|---|--------|--------|--------------------------|-----------------------------|
| 1 | Hidden | Output | 合計 RAM                   | 判定                          |
| 2 | 入力欄    | 入力欄    | = (B2+A2*2)*A2*2+16*1024 | =IF(C2 <= 131072,"OK","NG") |

図 3: 必要 RAM 容量を計算する Excel シートの例

入力データと出力データは学習・推論関数の StartTrain・StartPredict の引数に使用されているため、データの要素数がペイロードに収まる 4091~4093 以下のサイズでなければ事実上使用できない。

### 42h: ODL\_Reset

|     |       |             |     |     |    |
|-----|-------|-------------|-----|-----|----|
| 42h | 0001h | instance 1B | CRC | 戻り値 | なし |
|-----|-------|-------------|-----|-----|----|

指定したインスタンス番号の学習データをクリアする。

### 43h: ODL\_ResetReservoir

|     |       |             |     |     |    |
|-----|-------|-------------|-----|-----|----|
| 43h | 0001h | instance 1B | CRC | 戻り値 | なし |
|-----|-------|-------------|-----|-----|----|

隠れ層のリザーバーの状態をゼロにクリアする。

### 44h: ODL\_StartTrain

|     |     |             |                       |                        |     |     |    |
|-----|-----|-------------|-----------------------|------------------------|-----|-----|----|
| 44h | LEN | instance 1B | BF16 x 2B×(inputSize) | BF16 t 2B×(outputSize) | CRC | 戻り値 | なし |
|-----|-----|-------------|-----------------------|------------------------|-----|-----|----|

出力 y が教師 t と一致するように、入力 x を使って内部の学習データを更新・追加学習する。[Loss](#) は学習誤差となる。  
x と t のデータの総数は、CRC 不使用時 4093、CRC 使用時 4091 要素まで指定可能。

### 45h: ODL\_StartPredict

|     |     |             |                       |                        |     |     |    |
|-----|-----|-------------|-----------------------|------------------------|-----|-----|----|
| 45h | LEN | instance 1B | BF16 x 2B×(inputSize) | BF16 t 2B×(outputSize) | CRC | 戻り値 | なし |
|-----|-----|-------------|-----------------------|------------------------|-----|-----|----|

入力 x から出力 y を推論する。教師 t は出力 y との推論誤差 Loss を算出するために利用される。  
x と t のデータの総数は、CRC 不使用時 4093、CRC 使用時 4091 要素まで指定可能。

### 46h: ODL\_Transfer\_X

### 47h: ODL\_StartTrain\_T

### 48h: ODL\_StartPredict\_T

|     |     |             |                        |     |     |    |
|-----|-----|-------------|------------------------|-----|-----|----|
| 46h | LEN | instance 1B | BF16 x 2B×(inputSize)  | CRC | 戻り値 | なし |
| 47h | LEN | instance 1B | BF16 t 2B×(outputSize) | CRC | 戻り値 | なし |
| 48h | LEN | instance 1B | BF16 t 2B×(outputSize) | CRC | 戻り値 | なし |

[ODL\\_StartTrain](#) と [ODL\\_StartPredict](#) ではペイロードに入力 x と教師 t が含まれている。各々が大きくデータの総数がペイロードに収まらない場合、ODL\_Transfer\_X → ODL\_StartTrain\_T または ODL\_StartPredict\_T の順で分割転送すると、大きなサイズのデータを与えることができる。CRC 不使用なら 4093、CRC 使用なら 4091 要素まで指定可能。

ODL\_Transfer\_X は中間バッファへ転送される。同じデータは再利用できる？ 中間バッファは非破壊か不明。

## 9 コマンドリファレンス (続き)

### 49h: ODL\_GetResult

|     |       |     |     |     |     |                        |     |
|-----|-------|-----|-----|-----|-----|------------------------|-----|
| 49h | 0000h | CRC | 戻り値 | 01h | LEN | BF16 y 2B×(outputSize) | CRC |
|-----|-------|-----|-----|-----|-----|------------------------|-----|

直前に実行した StartTrain や StartPredict の出力値 y を取得する。

### 4Ah: ODL\_GetLoss

### 4Bh: ODL\_GetLossAsFloat

|     |       |             |     |     |     |       |               |     |
|-----|-------|-------------|-----|-----|-----|-------|---------------|-----|
| 4Ah | 0001h | instance 1B | CRC | 戻り値 | 01h | 0002h | BF16 Loss 2B  | CRC |
| 4Bh | 0001h | instance 1B | CRC | 戻り値 | 01h | 0004h | float Loss 4B | CRC |

StartTrain や StartPredict を実行後、引数に与えてあった教師 t と出力 y の学習誤差・推論誤差を [ODL Parameters 構造体](#) の lossFunction を使って算出する。

## 【OSUAD オンライン逐次学習型 教師なし学習コマンド】

現状の入力データ x を教師データ t として使用し、出力データ y へ複製するように学習させることを【教師なし学習】と呼び、前節の ODL コマンドが OSUAD コマンドとして使いやすいうに別名定義されている。

OSUAD 関数の動作の詳細がロームのアプリケーションノートに書かれていないため、使用する場合は動作をよく確認すること。

### 50h: OSUAD\_Initialize

|     |       |           |                    |     |     |    |
|-----|-------|-----------|--------------------|-----|-----|----|
| 50h | 0015h | models 1B | ODL_Parameters 20B | CRC | 戻り値 | なし |
|-----|-------|-----------|--------------------|-----|-----|----|

AI モデル インスタンスを指定した数だけ一括で初期化する。

事前に ODL\_SetModelCount でインスタンス数を宣言しておかなければいけない。

初期化するインスタンス数を 1~3 で指定すると、インスタンス 0 から指定した個数が初期化される。

インスタンスは、同一設定の ODL\_Parameters 構造体で初期化される。

AI エンジンのメモリ不足などで初期化が失敗した場合は、[コマンド実行エラー【12】](#)が戻る。

### 51h: OSUAD\_Reset

|     |       |     |     |    |
|-----|-------|-----|-----|----|
| 51h | 0000h | CRC | 戻り値 | なし |
|-----|-------|-----|-----|----|

OSUAD\_Initialize で初期化した全ての AI モデル インスタンスの学習データを一括でクリアする。

### 52h: OSUAD\_StartTrain

|     |     |             |                       |     |     |    |
|-----|-----|-------------|-----------------------|-----|-----|----|
| 52h | LEN | instance 1B | BF16 x 2B×(inputSize) | CRC | 戻り値 | なし |
|-----|-----|-------------|-----------------------|-----|-----|----|

入力 x を教師 t として利用し、出力 y が教師 t (すなわち入力 x) に近くなるように内部の学習データを更新・追加学習する。

[ODL\\_StartTrain](#) の t に x のデータを指定するのと同じ動作だが、教師データ t の転送を省略するのでデータ転送効率が良い。

### 53h: OSUAD\_StartPredict

|     |     |             |                       |     |     |    |
|-----|-----|-------------|-----------------------|-----|-----|----|
| 53h | LEN | instance 1B | BF16 x 2B×(inputSize) | CRC | 戻り値 | なし |
|-----|-----|-------------|-----------------------|-----|-----|----|

入力 x から出力 y を推論する。

入力 x を教師 t として利用し、出力 y と教師 t (すなわち入力 x) から推論誤差 Loss を算出する。

OSUAD\_StartTrain と同様に、教師データ t の転送を省略するのでデータ転送効率が良い。

## 9 コマンドリファレンス (続き)

### 54h: OSUAD\_GetLoss

|     |       |     |     |     |       |              |     |
|-----|-------|-----|-----|-----|-------|--------------|-----|
| 54h | 0000h | CRC | 戻り値 | 01h | 0002h | BF16 Loss 2B | CRC |
|-----|-------|-----|-----|-----|-------|--------------|-----|

[ODL\\_GetLoss](#) と同じく学習誤差・推論誤差を取得する。

ODL\_GetLoss の動作とは若干異なり、OSUAD\_Initialize で宣言したインスタンスの Loss の中で最も小さい値が戻る。

## 【ODL 重みデータセーブ・リストアコマンド】

重みデータ セーブ・リストアコマンドは、学習済みの重みデータを取得・復元することで、CPUリセットによる学習データの消失を防ぐことができる。重みデータのサイズは ODL\_Initialize や OSUAD\_Initialize で指定した [ODL\\_Parameters 構造体](#) によって変化し、学習を何度実行してもサイズは一定である。

### 60h: GetModels

|     |       |     |     |     |       |           |     |
|-----|-------|-----|-----|-----|-------|-----------|-----|
| 60h | 0000h | CRC | 戻り値 | 01h | 0001h | models 1B | CRC |
|-----|-------|-----|-----|-----|-------|-----------|-----|

ParaSol が管理している AI モデル インスタンス数を返す。

ODL\_SetModelCount の実行前は 0 が返り、実行後に models の値が更新される。

### 61h: GetMemoryUsage

|     |       |             |     |     |     |       |              |     |
|-----|-------|-------------|-----|-----|-----|-------|--------------|-----|
| 61h | 0001h | instance 1B | CRC | 戻り値 | 01h | 0014h | MemUsage 20B | CRC |
|-----|-------|-------------|-----|-----|-----|-------|--------------|-----|

指定したインスタンスのノード数、AI 処理に必要な RAM バイト数、重みデータの保存バイト数を MemUsage 構造体で返す。これらの値は ODL\_Initialize や OSUAD\_Initialize が実行されるたびに更新される。

MemUsage 構造体の内容を以下の表に示す。

表 5: MemUsage 構造体

| 項目             | 型      | バイト数 | 機能                     |
|----------------|--------|------|------------------------|
| inputSize      | uint16 | 2    | 入力ノード数。                |
| hiddenSize     | uint16 | 2    | 隠れ層ノード数。               |
| outputSize     | uint16 | 2    | 出力ノード数。                |
| AI RAM         | uint32 | 4    | AI 処理に必要な RAM のバイト数。   |
| $\beta$ Size   | uint32 | 4    | 学習重みデータ $\beta$ のバイト数。 |
| P Size         | uint32 | 4    | 追加学習用の重みデータ P のバイト数。   |
| Reservoir Size | uint16 | 2    | 隠れ層のフィードバック重みデータのバイト数。 |

### 62h: ODL\_GetParameters

|     |       |             |     |     |     |       |                    |     |
|-----|-------|-------------|-----|-----|-----|-------|--------------------|-----|
| 62h | 0001h | instance 1B | CRC | 戻り値 | 01h | 0014h | ODL_Parameters 20B | CRC |
|-----|-------|-------------|-----|-----|-----|-------|--------------------|-----|

指定したインスタンス番号の ODL\_Parameters 構造体を取得する。

## 9 コマンドリファレンス (続き)

### 63h: ODL\_GetWeightBeta

### 64h: ODL\_GetWeightP

|     |       |             |           |         |     |     |     |     |               |     |
|-----|-------|-------------|-----------|---------|-----|-----|-----|-----|---------------|-----|
| 63h | 0007h | instance 1B | offset 4B | size 2B | CRC | 戻り値 | 01h | LEN | $\beta$ (LEN) | CRC |
| 64h | 0007h | instance 1B | offset 4B | size 2B | CRC | 戻り値 | 01h | LEN | P (LEN)       | CRC |

学習重みデータ  $\beta$  および追加学習用の重みデータ P を取得する。

重みデータ  $\beta \cdot P$  のサイズは最大 112KB になるため、読み出し開始位置と読み出しサイズ指定して分割して取得する。

CRC 不使用なら 8188 バイト、CRC 使用なら 8184 バイトまで一度に取得可能。

ペイロードに入るサイズ以上の size を指定した場合は、サイズの値を自動で小さく調整する。

データの読み出し中にデータ終端に達する場合は、残りのデータ数に合わせて戻り値の LEN が調整される。

データ範囲外の読み出し開始位置を指定すると、戻り値のペイロード長は 0 になる。

つまり、offset = 0・最大サイズで読み出し、LEN が 0 になるまで繰り返し読むと、全てのデータが取得できる。

### 65h: ODL\_GetReservoir

|     |       |             |     |     |     |     |                          |     |
|-----|-------|-------------|-----|-----|-----|-----|--------------------------|-----|
| 65h | 0001h | instance 1B | CRC | 戻り値 | 01h | LEN | BF16 state 2B×hiddenSize | CRC |
|-----|-------|-------------|-----|-----|-----|-----|--------------------------|-----|

隠れ層のリザーバーの状態を取得する。データ数は hiddenSize と同一である。

### 66h: ODL\_SetWeightBeta

### 67h: ODL\_SetWeightP

|     |     |             |           |                  |     |     |    |
|-----|-----|-------------|-----------|------------------|-----|-----|----|
| 66h | LEN | instance 1B | offset 4B | $\beta$ (LEN-5B) | CRC | 戻り値 | なし |
| 67h | LEN | instance 1B | offset 4B | P (LEN-5B)       | CRC | 戻り値 | なし |

学習重みデータ  $\beta$  および追加学習用の重みデータ P を AI モデルに書き込む。

重みデータ  $\beta \cdot P$  のサイズは最大 112KB になるため、書き込み開始位置を指定して分割して書き込む。

CRC 不使用なら 8183 バイト、CRC 使用なら 8179 バイトまで一度に書き込み可能。

範囲外の書き込み開始位置やペイロード長を指定すると、データを書き込まずに[コマンド解析エラー【10】](#)を返す。

### 68h: ODL\_SetReservoir

|     |     |             |                          |     |     |    |
|-----|-----|-------------|--------------------------|-----|-----|----|
| 68h | LEN | instance 1B | BF16 state 2B×hiddenSize | CRC | 戻り値 | なし |
|-----|-----|-------------|--------------------------|-----|-----|----|

隠れ層のリザーバーの状態を書き込む。データ数は hiddenSize と同一である。



## 10 移植の手引き

この章では、ParaSol を AIBBY 以外のボードで使うための移植の方法について、ポート割り当ての観点で解説します。基板の改造はご自身の技量と自己責任で実施してください。

### AIBBY でのポート割り当て

以下に ParaSol のシングルポート構成時のポート割り当てを示します。

表 6: ParaSol のポート割り当て

| SPI                             | READY | LED  |
|---------------------------------|-------|------|
| SSIOF1                          | GPIO  | GPIO |
| P60:SCK P61:SDO P62:SDI P63:SS# | P76   | P72  |

SPI は SSIOF チャンネル 1 を使用しています。DMA や外部割り込みを併用するため、チャンネル 0 への変更は手間がかかります。READY は FT232H の GPIO でポーリングすることを想定し、LED と同じ P70 系統を使用しています。READY と LED 両方を監視すると、LED が消えて READY が High の場合に、省電力スタンバイ状態を検出できます。

### データ・テクノ製 DT-EBML63Q2557 評価ボード 内蔵 FT2232H 経由で PC と通信する例

※ この評価ボードはデュアルポート構成で動作させることができません。SDO の P33 ポートが故障する可能性があります。

SPI は AIBBY と同じポート、READY は未使用ピン P76 を使っているため、これらの機能のプログラム変更は不要です。READY を評価ボード内蔵の FT2232H GPIO bit0 でポーリングするには、TP27 から U17 の 26 番ピンへ追加配線してください。LED はプログラムを変更して P54～P56 のどれかに割り当てを変えてください。P56 にすると初期化周りの変更が楽です。LED のポートを変更しなかった場合でも、液晶画面のリセット信号が駆動されるだけで基板が故障することはありません。

この評価ボードで ParaSol を使用する場合は、これらの変更のほかに、電源保持信号 (P45 POWER\_KEEP) に High を出力するか、ジャンパーの JP8 を短絡して電源を強制的に ON にしないと、電源がすぐに切れてしまいます。

表 7: DT-EBML63Q2557 評価ボード 内蔵の FT2232H 経由で PC と通信する場合のポート割り当て例

| SPI                             | READY | LED          |
|---------------------------------|-------|--------------|
| 変更不要                            | 追加配線  | ポート変更        |
| P60:SCK P61:SDO P62:SDI P63:SS# | P76   | P54～P56 のどれか |

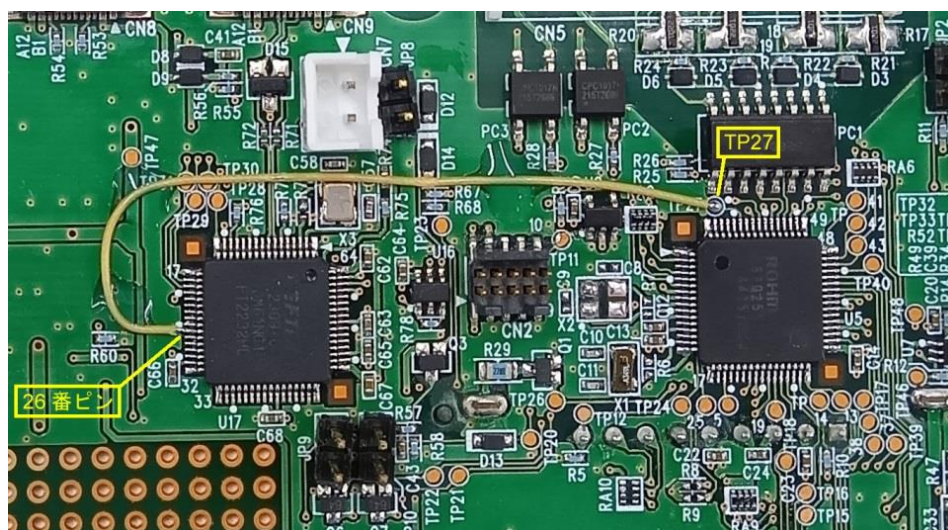


図 4: READY を FT2232H の GPIO bit0 へ入力するための追加配線

## 10 移植の手引き (続き)

### データ・テクノ製 DT-EBML63Q2557 評価ボード 黒いインターフェースコネクタを使う例

黒いインターフェースコネクタの SPI はチャンネル 0 が使われているので、SSIOF・DMA・外部割り込みの変更が必要です。READY と LED の両方を外部へ引き出す例を以下の表に示します。

表 8: DT-EBML63Q2557 評価ボード 横のインターフェースコネクタを使う場合のポート割り当て例

| SPI                                           | READY        | LED          |
|-----------------------------------------------|--------------|--------------|
| SSIOF0 へ変更<br>P40:SCK P41:SDO P42:SDI P43:SS# | ポート変更<br>P23 | ポート変更<br>P22 |

### ローム製 RB-D63Q2557TB64 リファレンスボード

このリファレンスボードでは、プログラムを変更せずに利用できます。デュアルポート構成で動作させることも可能です。CPU の信号が外部コネクタ CN2 に引き出されています。SPI の信号は CN3 から取り出すことも可能です。基板上の LED を使用する場合は、P52 を使うと初期化周りの変更が楽です。

### ML63Q253x 48 ピンパッケージデバイス

48 ピンデバイスにはポート P70 系統が存在しませんので、READY と LED の信号の割り当て変更が必要です。P50 系統を使用すると初期化周りのプログラム変更がやりやすく、基板パターンの引き回しが SPI の信号と近いのでお勧めです。デュアルポート構成も、問題なく動作します。

表 9: ML63Q253x 48 ピンパッケージデバイスで使用する場合のポート割り当て例

| SPI                                     | READY        | LED          |
|-----------------------------------------|--------------|--------------|
| 変更不要<br>P60:SCK P61:SDO P62:SDI P63:SS# | ポート変更<br>P56 | ポート変更<br>P52 |

## 11 実行時間の目安

フレーム受信完了後に READY が非アクティブになってから、再びアクティブになるまでの実行時間の目安を記載します。

| --- 条件 ---                            | --- 時間 --- | --- 条件 --- | --- 時間 --- |
|---------------------------------------|------------|------------|------------|
| Standby Wakeup..... (クロックの起動時間に左右される) | 390 ms     |            |            |
| System Reset..... (クロックの起動時間に左右される)   | 520 ms     |            |            |
| 00h: NOP.....                         | 0.017 ms   |            |            |
| 00h: NOP、CRC チェック、1024 バイトフレーム.....   | 0.17 ms    |            |            |
| 00h: NOP、CRC チェック、8191 バイトフレーム.....   | 1.06 ms    |            |            |
| 21h: FFT_Execute、512 入力、窓なし.....      | 4.2 ms     |            |            |
| 21h: FFT_Execute、512 入力、ハニング窓.....    | 4.4 ms     |            |            |
| 21h: FFT_Execute、1024 入力、窓なし.....     | 9.2 ms     |            |            |
| 21h: FFT_Execute、1024 入力、ハニング窓.....   | 9.6 ms     |            |            |
| 21h: FFT_Execute、2048 入力、窓なし.....     | 19.6 ms    |            |            |
| 21h: FFT_Execute、2048 入力、ハニング窓.....   | 20.4 ms    |            |            |
| 31h: FixedToBfloat16、1024 要素.....     | 1.4 ms     |            |            |
| 31h: FixedToBfloat16、2048 要素.....     | 2.6 ms     |            |            |
| 32h: Bfloat16ToFixed、1024 要素.....     | 1.1 ms     |            |            |
| 32h: Bfloat16ToFixed、2048 要素.....     | 2.1 ms     |            |            |
| 33h: Bfloat16ToFloat、1024 要素.....     | 0.31 ms    |            |            |
| 33h: Bfloat16ToFloat、2048 要素.....     | 0.59 ms    |            |            |

## 12 著作権・ライセンス・免責事項

### 【著作権・ライセンス】

本ソフトウェアおよび関連する文書のファイルの複製を取得した全ての人物に対し、下記枠内の著作権表示をソフトウェアの全ての複製または実質的な部分に記載することを条件に、ソフトウェアを無制限に扱うことを無償で許可します。

ParaSol - Converts the Solist-AI™ MCU into an SPI peripheral device.  
Copyright (c) 2026 Gadget Factory [http://park19.wakwak.com/~gadget\\_factory/](http://park19.wakwak.com/~gadget_factory/)

### 【免責事項】

ソフトウェアは「現状有姿」で提供され、いかなる種類の保証も行いません。著作者は、ソフトウェアまたはソフトウェアの使用もしくはその他に取り扱いに起因または関連して生じるいかなる請求、損害賠償、その他の責任について、一切の責任を負いません。

商用目的での製品化・収益化を考えておられる方は、ご自身の責任においてご使用ください。これはエラー処理の省略や意図せずに混入したバグなど、安全性の側面から是非とも守っていただきたいところです。アイデアやノウハウを利用していただくのは結構ですが、丸コピーで運用しようと考えておられる方は、十分ご検討をお願いします。

## 13 バグレポート

責任を負いませんとは言うものの、問題・改善提案などがあればメールでご連絡ください。できるだけ対応させていただきます。

>>> [gadget.factory.mail@gmail.com](mailto:gadget.factory.mail@gmail.com) <または> [gadget\\_factory@bf.wakwak.com](mailto:gadget_factory@bf.wakwak.com)

## 14 謝辞

Solist-AI™ の機能や仕様などの資料の提供と、ご助言をいただきました株式会社ロームの皆様には、心より感謝いたします。

## 15 バージョン履歴

| Ver. | 公開日        | 内容     |
|------|------------|--------|
| 0.0  | 2026-06-06 | 制作開始   |
| 1.0  | 2026-06-24 | 初回リリース |



## 16 目次

---

|                                    |    |
|------------------------------------|----|
| 1 概要.....                          | 1  |
| 2 特徴.....                          | 1  |
| 3 応用例.....                         | 1  |
| 4 動作概要.....                        | 1  |
| 【シングルポート構成】.....                   | 2  |
| 【デュアルポート構成】.....                   | 2  |
| 5 通信データのフレーム構造.....                | 2  |
| 6 フレーム構造の詳細.....                   | 3  |
| 【ParaSol が受信するフレーム】.....           | 3  |
| 【ParaSol から送信するフレーム】.....          | 3  |
| 7 結果コード一覧.....                     | 4  |
| 8 動作状態の判定方法.....                   | 4  |
| 9 コマンドリファレンス.....                  | 5  |
| 【コマンド大分類】.....                     | 5  |
| 【コマンド実行の注意点】.....                  | 5  |
| 【システムコマンド】.....                    | 5  |
| ----: System Reset.....            | 5  |
| 01h: CRC Check Off.....            | 5  |
| 02h: CRC Check On.....             | 5  |
| 03h: Set Idle Clock.....           | 6  |
| 04h: Enter Standby Mode.....       | 6  |
| 05h: Get Version.....              | 6  |
| 06h: Calc Payload CRC.....         | 6  |
| 07h: Readback Payload.....         | 6  |
| 08h: Change Access Port.....       | 6  |
| 【ML_ACC コマンド】.....                 | 7  |
| 10h: ML_ACC_ClearRAM.....          | 7  |
| 【FFT コマンド】.....                    | 7  |
| 20h: FFT_Initialize.....           | 7  |
| 21h: FFT_Execute.....              | 7  |
| 【数値変換コマンド】.....                    | 8  |
| 30h: ODL_GenerateRandomNumber..... | 8  |
| 31h: ODL_FixedToBfloat16.....      | 8  |
| 32h: ODL_Bfloat16ToFixed.....      | 8  |
| 33h: ODL_Bfloat16ToFloat.....      | 8  |
| 【ODL オンデバイス学習コマンド】.....            | 9  |
| 40h: ODL_SetModelCount.....        | 9  |
| 41h: ODL_Initialize.....           | 9  |
| 42h: ODL_Reset.....                | 10 |
| 43h: ODL_ResetReservoir.....       | 10 |
| 44h: ODL_StartTrain.....           | 10 |
| 45h: ODL_StartPredict.....         | 10 |
| 46h: ODL_Transfer_X.....           | 10 |
| 47h: ODL_StartTrain_T.....         | 10 |
| 48h: ODL_StartPredict_T.....       | 10 |
| 49h: ODL_GetResult.....            | 11 |

## 16 目次 (続き)

|                                                              |    |
|--------------------------------------------------------------|----|
| 4Ah: ODL_GetLoss .....                                       | 11 |
| 4Bh: ODL_GetLossAsFloat .....                                | 11 |
| 【OSUAD オンライン逐次学習型 教師なし学習コマンド】 .....                          | 11 |
| 50h: OSUAD_Initialize .....                                  | 11 |
| 51h: OSUAD_Reset .....                                       | 11 |
| 52h: OSUAD_StartTrain .....                                  | 11 |
| 53h: OSUAD_StartPredict .....                                | 11 |
| 54h: OSUAD_GetLoss .....                                     | 12 |
| 【ODL 重みデータセーブ・リストアコマンド】 .....                                | 12 |
| 60h: GetModels .....                                         | 12 |
| 61h: GetMemoryUsage .....                                    | 12 |
| 62h: ODL_GetParameters .....                                 | 12 |
| 63h: ODL_GetWeightBeta .....                                 | 13 |
| 64h: ODL_GetWeightP .....                                    | 13 |
| 65h: ODL_GetReservoir .....                                  | 13 |
| 66h: ODL_SetWeightBeta .....                                 | 13 |
| 67h: ODL_SetWeightP .....                                    | 13 |
| 68h: ODL_SetReservoir .....                                  | 13 |
| 10 移植の手引き .....                                              | 14 |
| AIBBY でのポート割り当て .....                                        | 14 |
| データ・テクノ製 DT-EBML63Q2557 評価ボード 内蔵 FT2232H 経由で PC と通信する例 ..... | 14 |
| データ・テクノ製 DT-EBML63Q2557 評価ボード 黒いインターフェースコネクタを使う例 .....       | 15 |
| ローム製 RB-D63Q2557TB64 リファレンスボード .....                         | 15 |
| ML63Q253x 48 ピンパッケージデバイス .....                               | 15 |
| 11 実行時間の目安 .....                                             | 15 |
| 12 著作権・ライセンス・免責事項 .....                                      | 16 |
| 13 バグレポート .....                                              | 16 |
| 14 謝辞 .....                                                  | 16 |
| 15 バージョン履歴 .....                                             | 16 |
| 16 目次 .....                                                  | 17 |

## 16 目次 (続き)

---

### 表

---

|                                                                     |    |
|---------------------------------------------------------------------|----|
| 表 1: 結果コードからわかる ParaSol の動作状態 .....                                 | 4  |
| 表 2: LED・READY 信号の状態 .....                                          | 4  |
| 表 3: 待機クロック速度 設定値 .....                                             | 6  |
| 表 4: ODL_Parameters 構造体 .....                                       | 9  |
| 表 5: MemUsage 構造体 .....                                             | 12 |
| 表 6: ParaSol のポート割り当て .....                                         | 14 |
| 表 7: DT-EBML63Q2557 評価ボード 内蔵の FT2232H 経由で PC と通信する場合のポート割り当て例 ..... | 14 |
| 表 8: DT-EBML63Q2557 評価ボード 横のインターフェースコネクタを使う場合のポート割り当て例 .....        | 15 |
| 表 9: ML63Q253x 48 ピンパッケージデバイスで使用する場合のポート割り当て例 .....                 | 15 |

### 図

---

|                                                     |    |
|-----------------------------------------------------|----|
| 図 1: 動作ブロック図 シングルポート構成 .....                        | 1  |
| 図 2: 動作ブロック図 デュアルポート構成 .....                        | 2  |
| 図 3: 必要 RAM 容量を計算する Excel シートの例 .....               | 10 |
| 図 4: READY を FT2232H の GPIO bit0 へ入力するための追加配線 ..... | 14 |